

Hooks & composition patterns

COMPLETE LEARNING GUIDE

What You'll Learn

Hooks let function components use state, side effects, and other React features without ever writing a class, and they replaced most of the older lifecycle-method and higher-order-component/render-prop patterns with a simpler, more composable model built on plain functions. This submodule covers the rules that make hooks work correctly, the two hooks responsible for the most real-world bugs (useEffect's dependency array and cleanup function, and the memoization hooks useMemo/useCallback), how to extract and share stateful logic through custom hooks, and useRef for values that need to persist across renders without triggering one. It closes with composition patterns — how custom hooks replace the older render-props and higher-order-component techniques for sharing logic between components.

1. The Rules of Hooks

Hooks must be called in the exact same order on every render of a given component — which means they can never be called conditionally, inside a loop, or after an early return; they must always be called at the top level of a function component or another hook. React relies on call order (not names) to associate each useState/useEffect call with its correct stored state between renders, so calling a hook conditionally would misalign that association on renders where the condition differs, corrupting state silently. The second rule — hooks can only be called from React function components or from other custom hooks, never from plain JavaScript functions — keeps this order-dependent bookkeeping working correctly.

JSX

```
// WRONG – hook called conditionally, breaks call-order consistency
function Profile({ userId }) {
  if (userId) {
    const [user, setUser] = useState(null); // violates rules of hooks
  }
}

// RIGHT – hook always called, condition handled inside
function Profile({ userId }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    if (userId) fetchUser(userId).then(setUser);
  }, [userId]);
}
```

2. useEffect: Dependency Arrays and Cleanup

`useEffect` synchronizes a component with something outside React's rendering model — fetching data, subscribing to an event, starting a timer. Its second argument, the dependency array, tells React when to re-run the effect: omitted entirely, it runs after every render; an empty array `[]` means it runs once after the initial mount only; a populated array `[a, b]` means it re-runs whenever any listed value changes between renders. Forgetting a value that the effect actually uses in the dependency array causes the effect to close over a stale version of that value, one of the single most common React bugs. An effect that subscribes to something or starts a timer should return a cleanup function, which React calls before the effect re-runs and when the component unmounts — omitting cleanup is how event listeners and intervals silently pile up as memory/performance leaks.

JSX

```
function LiveClock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setTime(new Date()), 1000);
    return () => clearInterval(id); // cleanup – prevents a leaked interval on
    unmount;
  }, []); // empty array: subscribe once on mount, clean up once on unmount

  return <p>{time.toLocaleTimeString()}</p>;
}
```

3. useMemo and useCallback: Memoization

`useMemo(fn, deps)` recomputes and caches an expensive derived value only when one of its dependencies changes, instead of on every render. `useCallback(fn, deps)` does the same thing for a function reference itself, which matters specifically when that function is passed as a prop to a child wrapped in `React.memo` — without `useCallback`, a new function reference is created every render, defeating the memoized child's shouldn't-re-render optimization even though the function's logic never actually changed. Both are optimization tools, not correctness tools, and reaching for them by default on every value/function adds cognitive overhead and a small computation/memory cost of their own without necessarily buying a measurable performance improvement — profile first, memoize what's actually shown to be expensive.

JSX

```
const ExpensiveList = React.memo(function ExpensiveList({ items, onSelect }) {
  console.log('ExpensiveList rendered');
  return items.map((i) => <div key={i.id} onClick={() => onSelect(i.id)}>{i.name}
</div>);
});

function Parent({ items }) {
  const [count, setCount] = useState(0);

  // Without useCallback, a NEW function is created every Parent render,
  // which would defeat ExpensiveList's React.memo optimization
  const handleSelect = useCallback((id) => console.log('selected', id), []);

  return (
    <>
      <button onClick={() => setCount((c) => c + 1)}>Unrelated: {count}</button>
      <ExpensiveList items={items} onSelect={handleSelect} />
    </>
  );
}
```

4. useRef: Persisting Values Without Re-Rendering

`useRef(initialValue)` returns a mutable `{ current: value }` object that persists for the component's entire lifetime without causing a re-render when it changes — unlike state, which always triggers a re-render on update. Its two main uses are holding a direct reference to a DOM node (attached via the `ref` attribute on a JSX element, useful for focusing an input or measuring an element) and holding any mutable value that needs to survive across renders but should never itself cause a re-render, like a timer ID or a 'previous value' for comparison.

JSX

```
function SearchInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // direct DOM access, no re-render triggered
  }, []);

  return <input ref={inputRef} type="text" />;
}
```

5. Custom Hooks: Extracting Reusable Stateful Logic

A custom hook is simply a JavaScript function whose name starts with `use` and that calls other hooks internally — this naming convention is how both React's tooling (the linter) and other developers recognize that the rules of hooks apply to it. Custom hooks are the modern replacement for the older higher-order-component and render-props patterns for sharing stateful logic between components: instead of wrapping a component to inject behavior, you extract the behavior into a hook and call it directly wherever it's needed, which is both easier to read (no wrapper indirection) and easier to compose (calling multiple custom hooks in one

component, versus nesting multiple higher-order components).

JSX

```
function useDebounceValue(value, delayMs) {
  const [debounced, setDebounce] = useState(value);

  useEffect(() => {
    const timeoutId = setTimeout(() => setDebounce(value), delayMs);
    return () => clearTimeout(timeoutId); // cancel the pending update if value
    // changes again
  }, [value, delayMs]);

  return debounced;
}

function SearchBox() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounceValue(query, 300);

  useEffect(() => {
    if (debouncedQuery) searchApi(debouncedQuery);
  }, [debouncedQuery]); // only fires 300ms after the user stops typing

  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}
```

6. Composition Patterns: Compound Components

Compound components (like the earlier Card.Header/Card.Body example) are a composition pattern where a set of components implicitly share state through Context internally, while presenting a clean, declarative JSX API to the consumer with no explicit prop wiring needed between the parts. A Tabs component might expose Tabs, Tabs.List, Tabs.Tab, and Tabs.Panel as related pieces that internally coordinate which tab is active via a shared Context, while the consumer just writes the nested JSX structure declaratively without manually tracking or passing the active index themselves.

Common Pitfalls

- Calling a hook conditionally or after an early return, breaking React's call-order-based state association.
- Omitting a value the effect actually reads from useEffect's dependency array, causing the effect to operate on a stale closure of that value.
- Omitting the cleanup function from an effect that subscribes to an event, timer, or socket, leaking listeners/timers across re-renders and unmounts.
- Wrapping every value in useMemo and every function in useCallback preemptively, adding overhead without a measured performance benefit.

- Naming a function that calls hooks internally without a 'use' prefix, so ESLint's hooks plugin can't verify it follows the rules of hooks.
- Using useRef to store a value that should actually drive a re-render (mutating ref.current expecting the UI to update, which it won't).

Key Takeaways

- Hooks must be called unconditionally, in the same order, on every render — no exceptions inside conditionals, loops, or early returns.
- An incomplete useEffect dependency array is the single most common source of stale-closure bugs in React.
- Effects that subscribe to something must return a cleanup function, or the subscription/timer/listener leaks across renders.
- useMemo/useCallback are performance tools to reach for after profiling, not defaults to apply to every value and function.
- A custom hook is just a function starting with 'use' that calls other hooks — the modern way to share stateful logic between components.

Learn More

React — Reference: Hooks

<https://react.dev/reference/react/hooks>

React — Synchronizing with Effects (useEffect)

<https://react.dev/learn/synchronizing-with-effects>

React — Reusing Logic with Custom Hooks

<https://react.dev/learn/reusing-logic-with-custom-hooks>